



Regressão linear com Python

Fatec 2026

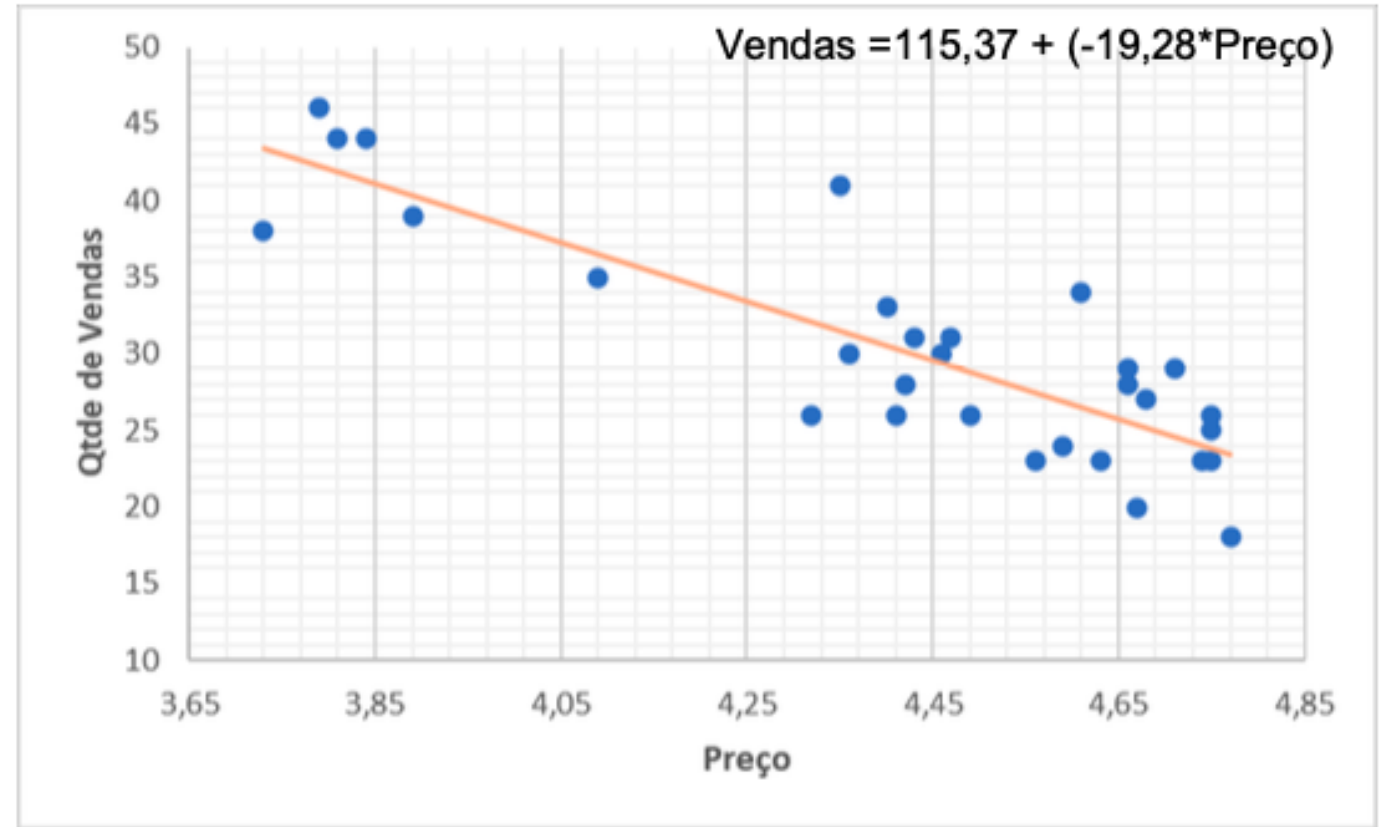
Regressão Linear Simples

Começamos com a relação entre **duas** variáveis: 1 Preditora (X) e 1 Resposta (Y).

Buscamos a linha que melhor descreve os dados, representada pela equação:

$$y = \beta_0 + \beta_1 x_1 + \varepsilon$$

Exemplo: Nota da Prova ~ Horas de Estudo



$$Vendas \text{ do Café} = 115,37 + (-19,28 * Preço \text{ do Café})$$

Onde 115,37 é o intercepto B_0 e -19,28 é o coeficiente angular B_1 . Ou seja, a cada real que aumenta no preço, as vendas caem, em média, em 19,28 unidades.

A regressão linear simples é quando temos apenas uma variável preditora e uma variável resposta

```
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Dados do Exemplo do Café

```
data = {
    'Preço': [3.7, 3.8, 4.0, 4.2, 4.5, 4.8, 5.0, 4.1, 4.6, 4.3],
    'Vendas': [45, 42, 38, 35, 30, 28, 25, 40, 32, 36]
}
df = pd.DataFrame(data)
```

2. Configuração do Modelo

```
y = df['Vendas']
# Adicionando a constante para o intercepto (Beta 0)
X = sm.add_constant(df['Preço'])
modelo = sm.OLS(y, X).fit()
```


3. Dispersão Real

```
plt.figure(figsize=(10, 5))
plt.scatter(df['Preco'], df['Vendas'], color='blue', label='Dados Reais', s=80)
plt.title('Passo 1: Observando a Dispersão dos Dados')
plt.xlabel('Preço do Café (R$)')
plt.ylabel('Quantidade de Vendas')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```

4. Modelo de Regressão

```
plt.figure(figsize=(10, 5))
plt.scatter(df['Preco'], df['Vendas'], color='blue', label='Dados Reais', s=80, alpha=0.5)
plt.plot(df['Preco'], modelo.predict(X), color='red', linewidth=2, label='Linha de Regressão')
plt.title(f'Passo 2: O Modelo de Regressão Linear\n$R^2$: {modelo.rsquared:.2f}\ne P-valor: {modelo.pvalues[1]:.4f}')
plt.xlabel('Preço do Café (R$)')
plt.ylabel('Quantidade de Vendas')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```

5. Exibição do Resumo Estatístico para análise do P-valor e R^2

```
print(modelo.summary())
```

```
# 6.ENTRADA DE DADOS E PREDIÇÃO ---
```

```
print("\n" + "="*30)
```

```
print("SIMULADOR DE PREVISÃO DE VENDAS")
```

```
print("="*30)
```

```
try:
```

```
    # Receber o preço do usuário
```

```
    preco_input = float(input("Digite o preço que deseja simular (ex: 4.45): "))
```

```
    # Criar a entrada para predição: [constante, preco_escolhido]
```

```
    # Lembrar: a constante deve ser sempre 1.0
```

```
    entrada_usuario = [1.0, preco_input]
```

```
    # Realizar a predição baseada no modelo treinado
```

```
    predicao = modelo.predict(entrada_usuario)
```

```
    print(f"\nResultado:")
```

```
    print(f"Para um preço de R$ {preco_input:.2f}, a previsão é vender {predicao[0]:.2f} cafés.")
```

```
except ValueError:
```

```
    print("Erro: Por favor, digite um número válido usando ponto como separador decimal.")
```

Statsmodels

Em Python, temos duas bibliotecas principais para regressão:

sklearn (Scikit-learn):

- Foco em **Predição** (Machine Learning).
- Ótima para `.fit()` e `.predict()`, mas não fornece resumos estatísticos detalhados.

statsmodels:

- Foco em **Inferência** (Análise Estatística).
- É a biblioteca que fornece p-valores, R^2 , erros padrão e testes estatísticos

A Função: sm.OLS



A Função: `sm.OLS`

A função que usamos é a sm.OLS.

'OLS' significa **Ordinary Least Squares** (método dos Mínimos Quadrados), que é o método matemático usado para encontrar a "melhor linha" (ou hiperplano) que minimiza os erros.



A Filosofia

statsmodels não usa a sintaxe de fórmula ($y \sim x_1 + x_2$).

Força a construção das matrizes X (preditores) e y (resposta) separadamente

Código: Passo 1 – Preparando os Dados

Preparar X e y

Primeiro, separamos nosso DataFrame em duas partes:

y (Dependente): A coluna que queremos prever (ex: 'nota_prova').

X (Independentes): O DataFrame com todas as colunas que usamos para prever (ex: 'horas_estudo', 'horas_sono').

sm.add_constant

Deve-se adicionar manualmente uma coluna de "constante" à variável X. (independente) com `sm.add_constant(X)`

Código: Passo 2 – Treinando e Resumindo

`.fit()` e `.summary()`

O processo :

`sm.OLS(y, X)`: Define o modelo (quem é 'y', quem é 'X').

`.fit()`: Treina o modelo (encontra os coeficientes beta).

`.summary()`: Gera o resumo estatístico completo.

```
import statsmodels.api as sm
import pandas as pd

# Suponha que 'dados' é seu DataFrame
# dados = pd.read_csv(...)

# 1. Preparar y (dependente)
y = dados['nota_prova']

# 2. Preparar X (independentes)
X = dados[['horas_estudo', 'horas_sono']]

# 3. Adicionar a constante (intercepto)
X = sm.add_constant(X)

# 4. Definir e Treinar o modelo
modelo = sm.OLS(y, X).fit()

# 5. Gerar o resumo
print(modelo.summary())
```

O Resumo - summary()

O .summary() gera uma tabela. A parte mais importante é a tabela de coeficientes:

Variável	coef	std err	t	P> t	[0.025	0.975]
const	5.0123	0.500	10.024	0.000	4.000	6.024
horas_estudo	2.1050	0.200	10.525	0.000	1.699	2.511
horas_sono	0.8010	0.301	2.661	0.012	0.190	1.412

coef (Coeficiente): O impacto de cada variável. "Para cada 1h extra de estudo, a nota sobe 2.1 pontos, mantendo o sono constante."

P>|t| (P-valor): A significância estatística. Se < **0.05**, a variável é relevante para o modelo.

•**T (t-estatístico)**: Quanto maior o valor de t (seja positivo ou negativo), mais confiantes estamos de que a variável é importante. Quanto mais próximo de zero, estamos vendo seja apenas ruído nos dados.

0.025 e 0.975: Margem de erro padrão que os softwares de Ciência de Dados usam para garantir que os resultados não foram apenas fruto do acaso.

Conclusão: No nosso exemplo, 'horas_estudo' (p=0.000) e 'horas_sono' (p=0.012) são ambos preditores estatisticamente significativos.

O Código é bom? R^2

R^2 (R-Quadrado) é a métrica que diz quanto o seu modelo consegue explicar os dados
Ele varia de **0 a 1** (ou 0% a 100%)

- **O que ele mede:** A porcentagem da variação da variável Y (ex: Notas) que é explicada pelas variáveis X
 - (ex: Estudo e Sono).
- **A Regra de Ouro:**
 - **Próximo de 1.0 (100%):** O modelo é excelente
 - Os pontos estão muito perto da linha/plano de regressão
 - **Próximo de 0.0 (0%):** O modelo é ruim
 - As variáveis escolhidas não explicam nada do que está acontecendo

Exemplo Prático:

Se o seu $R^2 = 0.85$, você pode dizer que *"85% da variação nas vendas é explicada pelo preço que praticamos e pelas promoções que fazemos"*.

Mas o mundo é complexo.

A nota de um aluno depende **apenas** das horas de estudo?

E quanto a...

- Horas de Sono?
- Número de Faltas?
- Qualidade do professor?

A regressão simples não consegue isolar o efeito de uma variável enquanto controla as outras.

O que é Regressão Múltipla?



Definição

Uma extensão da regressão linear que usa **múltiplas** variáveis independentes (X) para prever uma única variável dependente (Y).



A Equação

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon$$

O modelo encontra os "pesos" (coeficientes) para cada variável X.



Exemplo

Prever a nota com base em múltiplos fatores:

`Nota ~ Horas_Estudo +
Horas_Sono + Faltas`

Para que serve?

O poder da regressão múltipla está na **inferência**:

Permite **isolar o efeito** de uma variável, controlando o efeito das outras. (Ex: "Qual o impacto de +1h de estudo, *mantendo as horas de sono constantes?*")

Cria modelos de **previsão mais precisos**, pois utilizam mais informações do mundo real.

Ajuda a identificar e **controlar por variáveis de confusão** que poderiam distorcer seus resultados.

Sorvete e Afogamento

- **Observação:** Um pesquisador coleta dados e descobre uma correlação positiva muito forte entre **vendas de sorvete (X)** e o **número de afogamentos (Y)**
- **Conclusão Falsa (Regressão Simples):** O pesquisador conclui que "vender sorvete causa afogamentos".
- **A Variável de Confusão (Z):** A variável que foi ignorada é a **Temperatura** (ou a estação, "Verão")

•Fazendo uma Regressão Simples:

```
•modelo = sm.OLS(afogamentos,  
sorvetes).fit()
```

- O modelo diria que "sorvetes" é um ótimo previsor (p-valor baixo, R^2 alto), o que está estatisticamente correto, mas causalmente errado

•Fazendo uma Regressão Múltipla:

```
•X = sm.add_constant(dados[['sorvetes',  
'temperatura']])
```

```
•modelo = sm.OLS(afogamentos, X).fit()
```

O resultado do `summary()` agora mostraria:

- temperatura: p-valor muito baixo (ex: 0.000) -> Sim, esta variável é importante
- sorvetes: p-valor muito alto (ex: 0.850) -> Não, esta variável não tem efeito real

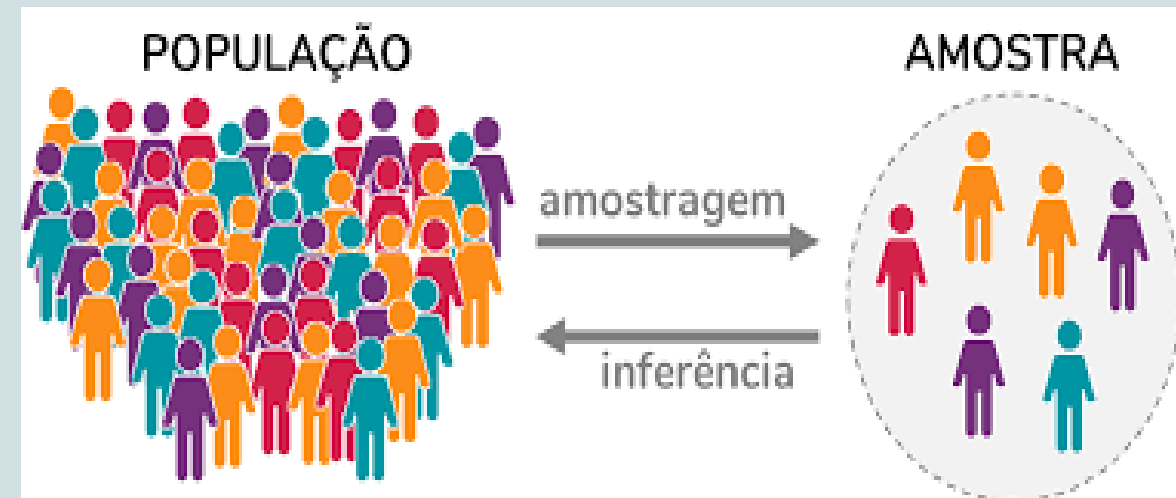
O que é Inferência?

Inferência Estatística é o processo de usar dados de uma **amostra** para tirar conclusões ou tomar decisões sobre uma **população** inteira.

População: O grupo completo que queremos estudar (ex: 150 milhões de eleitores no Brasil).

Amostra: Um subconjunto pequeno e gerenciável dessa população (ex: 2.000 eleitores entrevistados).

A inferência nos permite "dar um salto" da amostra para a população, usando a matemática para medir nossa incerteza.



Exemplo: Notas e tempo de estudo



A População

Definição: TODOS os alunos (passados, presentes e futuros) da FATEC.

O Problema: É impossível medir todos eles. Nós nunca saberemos o valor **verdadeiro** da relação entre Estudo e Nota para a população inteira.



A Amostra

Definição: Os 30 alunos que foram simulados no código.

O Cálculo: Na **amostra**, calculamos uma inclinação de 2.3082
Este é o nosso "dado observado"

*"A inclinação de 2.3082 que eu vi na minha amostra de 30 alunos foi apenas **sorte (acaso)**, ou ela representa uma **relação real** que também existe na população inteira?"*

A Resposta: Nosso P-valor foi **0.0000**. Isso significa que a chance de ser "sorte" é praticamente zero. Portanto, nós **inferimos** que a relação é real e existe na população.

Projeto Cafeteria : Estatística descritiva

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Criar o DataFrame ---
# um dicionário para criar o DataFrame do pandas
dados_dict = {
    'Vendas_Cafe': [18, 20, 23, 23, 23, 23, 24, 25, 26, 26, 26, 26, 27, 28, 28,
                    29, 29, 30, 30, 31, 31, 33, 34, 35, 38, 39, 41, 44, 44, 46],
    'Preco_Cafe': [4.77, 4.67, 4.75, 4.74, 4.63, 4.56, 4.59, 4.75, 4.75, 4.49,
                  4.41, 4.32, 4.68, 4.66, 4.42, 4.71, 4.66, 4.46, 4.36, 4.47, 4.43,
                  4.4, 4.61, 4.09, 3.73, 3.89, 4.35, 3.84, 3.81, 3.79],
    'Promocao': ["Nao", "Nao", "Nao", "Nao", "Nao", "Nao", "Nao", "Nao", "Sim",
                 "Nao", "Sim", "Nao", "Nao", "Sim", "Sim", "Nao", "Sim", "Sim",
                 "Sim", "Nao", "Nao", "Sim", "Sim", "Sim", "Nao", "Sim", "Sim",
                 "Sim", "Sim", "Sim"],
    'Preco_Leite': [4.74, 4.81, 4.36, 4.29, 4.17, 4.66, 4.73, 4.11, 4.21, 4.25,
                   4.62, 4.53, 4.44, 4.19, 4.37, 4.29, 4.57, 4.21, 4.77, 4, 4.31,
                   4.34, 4.05, 4.73, 4.07, 4.75, 4, 4.15, 4.34, 4.15]
}

dados = pd.DataFrame(dados_dict)
print("--- Primeiras 5 linhas dos Dados ---")
print(dados.head())
```

Projeto Cafeteria : Estatística descritiva

```
# --- Estatísticas Descritivas ---
# 'include="all"' garante que colunas não-numéricas (como Promocao) também sejam analisadas.
print("\n--- Estatísticas Descritivas (summary) ---")
print(dados.describe(include='all'))
# --- Desvio Padrão ---
print("\n--- Desvio Padrão (sd) ---")
print(f"Desvio Padrão Vendas Café: {dados['Vendas_Cafe'].std():.4f}")
print(f"Desvio Padrão Preço Café: {dados['Preco_Cafe'].std():.4f}")
print(f"Desvio Padrão Preço Leite: {dados['Preco_Leite'].std():.4f}")
```

Projeto Cafeteria : Como estão os dados?

```
# Histogramas ---
# 0 pandas pode criar histogramas para todas as colunas numéricas de uma vez
print("\n--- Gerando Histogramas ---")
# Criamos uma figura e eixos separados para os histogramas
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
# Histograma das Vendas
axes[0, 0].hist(dados['Vendas_Cafe'], color='blue', edgecolor='black')
axes[0, 0].set_title('Distribuicao das Vendas do Café')
axes[0, 0].set_xlabel('Vendas')
axes[0, 0].set_ylabel('Frequência')
# Histograma do Preço do Café
axes[0, 1].hist(dados['Preco_Cafe'], color='blue', edgecolor='black')
axes[0, 1].set_title('Distribuicao dos Preços do Café')
axes[0, 1].set_xlabel('Preço')
# Histograma do Preço do Leite
axes[1, 0].hist(dados['Preco_Leite'], color='blue', edgecolor='black')
axes[1, 0].set_title('Distribuicao dos Preços do Leite')
axes[1, 0].set_xlabel('Preço')
# Esconde o último gráfico (2,2) que está vazio
axes[1, 1].axis('off')
plt.tight_layout() # Ajusta o layout
plt.show() # Mostra o gráfico (equiv. a dev.off())
```

Projeto Cafeteria: Como estão as vendas? Preço influencia as vendas?

```
# --- Gráfico de Dispersão (Preço vs Vendas) ---
print("\n--- Relação Preço vs Vendas ---")
plt.figure(figsize=(10, 6))
plt.scatter(x=dados['Preco_Cafe'], y=dados['Vendas_Cafe'], color='blue')
plt.xlabel('Preço') # xlab
plt.ylabel('Quantidade Vendida') # ylab
plt.title('Relação entre o Preço e as Vendas do Café') # main
plt.grid(True) # grid()
plt.show()
```

Projeto Cafeteria: Promoções, fazem a diferença?

```
#Gráfico de Dispersão com Promoção ---
# Usar a biblioteca 'seaborn' (sns) é o jeito mais "pitônico"
# de plotar com cores baseadas em uma categoria (hue='Promocao')
# A legenda é criada automaticamente.
print("\n--- Relação Preço vs Vendas (com Promoção) ---")
plt.figure(figsize=(10, 6))
sns.scatterplot(data=dados,
                x='Preco_Cafe',
                y='Vendas_Cafe',
                hue='Promocao', # col = as.factor(dados$Promocao)
                style='Promocao', # pch = 16 (diferencia os marcadores)
                s=100) # Tamanho do marcador

plt.xlabel('Preço')
plt.ylabel('Quantidade Vendida')
plt.title('Relação entre o Preço e as Vendas do Café')
plt.grid(True)
plt.show()
```

Projeto Cafeteria: Quantas vezes as vendas ficaram acima ou abaixo da média?

```
# Variável Acima/Abaixo da Média ---
print("\n--- Análise Acima/Abaixo da Média ---")
media = dados['Vendas_Cafe'].mean()
# variavel <- ifelse(...) usada np.where para criar a nova coluna
dados['variavel'] = np.where(dados['Vendas_Cafe'] > media, 'Acima_da_media', 'Abaixo_da_media')
# table(variavel)
print("Contagem de Vendas Acima/Abaixo da Média:")
print(dados['variavel'].value_counts())
dados['variavel'].value_counts().plot(kind='bar', color=['green', 'red'], edgecolor='black')
plt.title('Contagem Acima vs. Abaixo da Média')
plt.ylabel('Contagem')
plt.xticks(rotation=0) # Deixa os rótulos do eixo X na horizontal
plt.show()
```

Np.Where é uma função que opera em *arrays* (como colunas de DataFrame) e permite fazer uma escolha "elemento a elemento" com base numa condição. Forma de se fazer uma operação "se-então-senão" em arrays sem a necessidade de um loop `for`.

Projeto Cafeteria: comparando vendas com e sem promoção

```
# Boxplot Vendas vs. Promoção ---
print("\n--- Gerando Boxplot: Vendas por Status da Promoção ---")
plt.figure(figsize=(8, 7))
# x = variável categórica (agrupador)
# y = variável numérica (o que estamos medindo)
sns.boxplot(data=dados,
            x='Promocao',
            y='Vendas_Cafe',
            palette=["#ff6347", "#90ee90"])
plt.title('Comparação de Vendas: Com vs. Sem Promoção', fontsize=16)
plt.xlabel('Promoção Ativa?', fontsize=12)
plt.ylabel('Vendas (Quantidade)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Projeto Cafeteria: Regressão Linear

```
# 1. Definir a fórmula
formula = 'Vendas_Cafe ~ Preco_Cafe'
# 2. Criar o modelo
# 'ols' = Ordinary Least Squares (Mínimos Quadrados Ordinários)
modelo = smf.ols(formula=formula, data=dados)
# 3. Treinar o modelo
resultados = modelo.fit()
# 4. Exibimos o resumo (equivalente ao summary(modelo))
print(resultados.summary())

# --- Gráfico de Regressão ---
print("Gerando gráfico de regressão (Dispersão + Linha)...")
plt.figure(figsize=(10, 6))
# A função regplot() (Regression Plot) do Seaborn
# plota o gráfico de dispersão (scatter plot) E
# calcula/plota a linha de regressão linear (regression line)
sns.regplot(data=dados,
            x='Preco_Cafe',
            y='Vendas_Cafe',
            scatter_kws={'color': 'blue'},
            line_kws={'color': 'red'})
plt.title('Relação entre o Preço e as Vendas do Café', fontsize=16)
plt.xlabel('Preço do Café', fontsize=12)
plt.ylabel('Vendas de Café', fontsize=12)
plt.grid(True)
# Mostra o gráfico
plt.show()
```

Projeto Cafeteria: Regressão linear Múltipla

```
#para clareza, eficiencia de memoria e robustez do código,  
#converte promocao para uma variável categorica  
dados['Promocao'] = dados['Promocao'].astype('category')  
formula_multipla = 'Vendas_Cafe ~ Preco_Cafe + Promocao + Preco_Leite'  
# Usamos smf.ols (Ordinary Least Squares)  
modelo_multipl = smf.ols(formula=formula_multipla, data=dados)  
# Treinamos o modelo  
resultados_multiplos = modelo_multipl.fit()  
print("--- Resultados da Regressão Múltipla (summary) ---")  
print(resultados_multiplos.summary())
```

- Sem a conversão, as categorias como texto comum (strings), o que **impede muitos tipos de análise estatística ou plotagem automática**
- Com a conversão, o sistema entende que "SIM" e "NÃO" são categorias distintas e não valores contínuos

- Statsmodels (através do pandas) é inteligente
- Na fórmula `Vendas_Cafe ~ ... + Promocao`, ela vê que a coluna "Promocao" contém texto (o dtype "object").
- Ela infere que esta é uma variável categórica e, na maioria dos casos, criaria as "variáveis dummy"

- Uma **variável dummy** (ou **variável indicadora**) é uma variável numérica binária que representa categorias qualitativas com **valores 0 ou 1**.
- É usada em modelos estatísticos, como **regressão linear múltipla**, para **incluir variáveis categóricas** (como sexo, cor, região) na análise, já que esses modelos requerem variáveis numéricas

Projeto Cafeteria: criar gráfico 3D

```
# coeficientes do modelo
intercepto = modelo_3d.params['Intercept']
coef_preco_cafe = modelo_3d.params['Preco_Cafe']
coef_preco_leite = modelo_3d.params['Preco_Leite']
# Criando a "grade" para o plano 3D
# 10 pontos espaçados entre o min e max de cada eixo X
x_surf = np.linspace(dados_completos['Preco_Cafe'].min(), dados_completos['Preco_Cafe'].max(), 10)
y_surf = np.linspace(dados_completos['Preco_Leite'].min(), dados_completos['Preco_Leite'].max(), 10)
# np.meshgrid cria a grade 2D
x_surf, y_surf = np.meshgrid(x_surf, y_surf)
# Calculando os valores Z (vendas) para o plano usando a equação da regressão
z_surf = intercepto + (coef_preco_cafe * x_surf) + (coef_preco_leite * y_surf)
# --- Configurando o Gráfico 3D ---
print("\nGerando gráfico 3D (interativo)...")
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
# Plotar os pontos (gráfico de dispersão 3D)
ax.scatter(dados_completos['Preco_Cafe'],
           dados_completos['Preco_Leite'],
           dados_completos['Vendas_Cafe'],
           c='blue', marker='o', label='Dados Reais')
# Plotar o plano de regressão (superfície)
# alpha=0.4 torna o plano semitransparente
ax.plot_surface(x_surf, y_surf, z_surf, color='red', alpha=0.4, label='Plano de Regressão')
ax.set_xlabel('Preço do Café (X1)')
ax.set_ylabel('Preço do Leite (X2)')
ax.set_zlabel('Vendas de Café (Y)')
ax.set_title('Regressão Múltipla 3D (Vendas ~ Preço Café + Preço Leite)', fontsize=16)
plt.show()
```

Projeto Cafeteria : comparando efeitos marginais

```
dados = pd.DataFrame(dados_dict)
# Converter Promocao para categoria
dados['Promocao'] = dados['Promocao'].astype('category')
# ---Modelo Completo (Com Promocao) ---
formula_completa = 'Vendas_Cafe ~ Preco_Cafe + Promocao + Preco_Leite'
modelo_completo = smf.ols(formula=formula_completa, data=dados).fit()
print("--- Resultados do Modelo Completo (Com Promoção) ---")
print(modelo_completo.summary())

#Modelo Reduzido (Sem Promocao) ---
formula_sem_promocao = 'Vendas_Cafe ~ Preco_Cafe + Preco_Leite'
modelo_sem_promocao = smf.ols(formula=formula_sem_promocao, data=dados).fit()
print("\n--- Resultados do Modelo Reduzido (Sem Promoção) ---")
print(modelo_sem_promocao.summary())
```

Projeto Cafeteria : Teste **ANOVA** (Análise de Variância)

```
# Comparação dos Modelos (ANOVA) ---
print("\n--- Teste ANOVA (Comparação dos Modelos) ---")
# Testamos se o modelo_completo é estatisticamente melhor que o modelo_sem_promocao
# Um P-valor (PR(>F)) baixo significa que a 'Promocao' é uma variável importante.
anova = sm.stats.anova_lm(modelo_sem_promocao, modelo_completo)
print(anova)
```

A função `sm.stats.anova_lm` é o **Teste ANOVA**

para comparar dois modelos de regressão aninhados (um modelo "simples" contra um modelo "complexo")

Ao comparar um modelo sem a Promoção com um modelo com a Promoção, o ANOVA testa se a variância extra explicada pela variável Promoção é grande o suficiente para ser considerada que ela melhora o modelo

```
# Gráficos de Efeitos Marginais (Partial Regression Plots) ---
print("\nGerando gráficos de Efeitos Marginais (do modelo completo)...")
# sm.graphics.plot_partregress_grid plota o efeito de cada variável controlando pelas outras.
fig = sm.graphics.plot_partregress_grid(modelo_completo)
fig.tight_layout()
plt.show()
```

Questões e análise do Projeto

- O preço do café influencia a venda?
- Promoções fazem diferenças?
- Quantas vezes as vendas ficaram acima/abaixo da média?
- Como é a relação entre a variável independente e as vendas? É crescente, decrescente ou curva?
- Há uma faixa da variável independente onde a previsão de vendas é menos confiável?
- Qual a variável tem maior efeito nas vendas?
- Analise o erro residual e o R^2 : os valores são bons ou ruins? Compare o resultado da regressão simples com a múltipla. Qual o melhor modelo?

O que você sugere para a cafeteria manter/ampliar as vendas de café?

Projeto Floricultura



- Você foi contratado por uma floricultura especializada em espécies do gênero *Iris*. A empresa deseja entender como diferentes características das flores se relacionam entre si, a fim de prever o **comprimento das pétalas** a partir de outras medições físicas.
- Para isso, você terá acesso a uma base de dados clássica da estatística chamada **iris**, que contém medições de 150 flores de três espécies (*setosa*, *versicolor* e *virginica*). Cada flor foi medida em relação a:
 - **Sepal.Length** – comprimento da sépala (em cm)
 - **Sepal.Width** – largura da sépala (em cm)
 - **Petal.Length** – comprimento da pétala (em cm)
 - **Petal.Width** – largura da pétala (em cm)
 - **Species** – espécie da flor

Projeto floricultura

- Qual variável teve o maior impacto sobre o comprimento da pétala: o comprimento da sépala ou a largura da sépala?
- O modelo parece se ajustar bem visualmente aos dados?
- Como você interpretaria esse modelo para um colega da floricultura que não conhece estatística?

Projeto floricultura – extração transformação e carga

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm
import statsmodels.formula.api as smf
# Carregar e Preparar os Dados ---
iris = load_iris()
# Converter para DataFrame do Pandas para facilitar o manuseio
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
# Renomear colunas para remover espaços e (cm), facilitando o uso
# em fórmulas (especialmente para statsmodels)
df.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
# Adicionar o target (espécie) ao dataframe, embora não usemos na regressão
df['species'] = iris.target_names[iris.target]
print("--- Amostra dos Dados (DataFrame) ---")
print(df.head())
print("-" * 40)
```

Projeto floricultura – regressão simples

```
# Regressão Simples (Exemplo: sepal_length vs petal_length) ---
print("\n--- 2. Regressão Linear Simples ---")
# Variável independente (X)
X_simple = df[['sepal_length']]
# Variável dependente (Y)
y = df['petal_length']

model_simple = LinearRegression()
model_simple.fit(X_simple, y)
y_pred_simple = model_simple.predict(X_simple)

print(f"Coeficiente (Inclinação): {model_simple.coef_[0]:.4f}")
print(f"Intercepto: {model_simple.intercept_:.4f}")

# Gráfico da Regressão Simples
plt.figure(figsize=(10, 6))
plt.scatter(X_simple, y, color='blue', label='Dados Reais')
plt.plot(X_simple, y_pred_simple, color='red', linewidth=2, label='Linha de Regressão')
plt.title('Regressão Simples: Comprimento da Sépala vs. Comprimento da Pétala')
plt.xlabel('Comprimento da Sépala (sepal_length)')
plt.ylabel('Comprimento da Pétala (petal_length)')
plt.legend()
plt.grid(True)
plt.show()
```

Projeto Floricultura – modelo regressão múltipla

```
# Regressão Múltipla (Usando Statsmodels para análise detalhada) ---
# Y = petal_length
# X1 = sepal_length
# X2 = sepal_width
print("\n--- 3. Regressão Linear Múltipla (Statsmodels) ---")
# A fórmula 'Y ~ X1 + X2' é uma forma padrão de definir o modelo
# statsmodels cuida de adicionar o intercepto automaticamente
model_multi_sm = smf.ols(formula='petal_length ~ sepal_length + sepal_width', data=df).fit()
# Exibir o sumário estatístico completo
print(model_multi_sm.summary())
```

Projeto Floricultura – Gráfico 3D

```
# Gráfico 3D da Regressão Múltipla ---
# modelo sklearn para prever facilmente a superfície do plano
X_multi = df[['sepal_length', 'sepal_width']]
model_multi_sk = LinearRegression()
model_multi_sk.fit(X_multi, y)
# Criar uma "grade" de pontos para desenhar o plano
x_surf, y_surf = np.meshgrid(
    np.linspace(df['sepal_length'].min(), df['sepal_length'].max(), 100),
    np.linspace(df['sepal_width'].min(), df['sepal_width'].max(), 100)
)
# Prever o valor Z (petal_length) para cada ponto da grade
z_surf = model_multi_sk.predict(np.c_[x_surf.ravel(), y_surf.ravel()]).reshape(x_surf.shape)
# Criar o gráfico 3D
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
# Plotar os dados reais (pontos)
ax.scatter(df['sepal_length'], df['sepal_width'], df['petal_length'], c='blue', marker='o', label='Dados Reais')
# Plotar o plano de regressão
ax.plot_surface(x_surf, y_surf, z_surf, color='red', alpha=0.4, label='Plano de Regressão')
ax.set_title('Regressão Múltipla 3D')
ax.set_xlabel('Comprimento da Sépala (sepal_length)')
ax.set_ylabel('Largura da Sépala (sepal_width)')
ax.set_zlabel('Comprimento da Pétala (petal_length)')
ax.view_init(elev=20, azim=45) # Ajustar o ângulo de visão
plt.show()
```

Projeto Floricultura: comparando modelos

```
# --- 4. Teste ANOVA ---  
# A tabela ANOVA mostra a contribuição de cada variável para o modelo  
print("\n--- 4. Tabela ANOVA ---")  
anova_table = sm.stats.anova_lm(model_multi_sm, typ=2)  
print(anova_table)
```

Projeto Floricultura: onde encontrar as respostas?

- Qual variável teve o maior impacto sobre o comprimento da pétala: o comprimento da sépala ou a largura da sépala?
 - No sumário, observe coef e a coluna T ou F no Anova
- O modelo parece se ajustar bem visualmente aos dados?
 - Analise se os pontos azuis (dados reais) se distribuem ao redor do plano vermelho (dados do modelo)
- Como você interpretaria esse modelo para um colega da floricultura que não conhece estatística?

"sépalas mais longas significam pétalas muito mais ____" e
"sépalas mais largas significam pétalas um pouco mais
____".

Atividade: concessionária

- Você trabalha em uma **concessionária de carros** e deseja entender como características técnicas dos veículos influenciam o **consumo de combustível (mpg - miles per gallon)**
- Você tem à disposição a base de dados mtcars, que contém informações de 32 modelos de carro, com variáveis como:
 - mpg: consumo (milhas por galão)
 - hp: potência do motor (horsepower)
 - wt: peso do carro
 - cyl: número de cilindros
 - am: tipo de transmissão (0 = automática, 1 = manual)

```
import pandas as pd
import statsmodels.api as sm

mtcars_df = sm.datasets.get_rdataset('mtcars').data
print(mtcars_df.head())
```


Atividade

- **Parte 0 – Estatística descritiva**
 - Use a estatística descritiva para explorar a base.
 - Gere gráficos de dispersão para sua visualização dos dados
- **Parte 1 – Regressão Linear Simples**
 - Ajuste um modelo de regressão linear simples prevendo **mpg** a partir de **hp**
- **Parte 2 – Regressão Linear Múltipla**
 - Adicione o **peso** do carro como segunda variável

Atividade

- **Parte 3 – gráfico 3D**

- Existe algum ponto extremo (outlier) que parece não se ajustar bem à reta/plano de regressão?

- **Parte 4 – Interpretação**

- Responda:

- O que significa o sinal dos coeficientes?
- O modelo com duas variáveis é melhor que o modelo simples? Justifique usando o anova()
- Que variáveis poderiam ser incluídas para melhorar o modelo?